



Институт экономики и управления
Кафедра статистики и кибернетики

Направление: 09.03.02 «Информационные системы и технологии»
Направленность: «Компьютерные науки и информационный анализ данных»
Методы и средства проектирования информационных систем и технологий

КУРСОВОЙ ПРОЕКТ

на тему: «Разработка информационной системы учета продуктов питания с анализом чеков и контролем сроков годности».

Выполнила:
Студент 3 курса группы Д-Э 341
Нгуен Фьонг Ань

Дата регистрации КП
на кафедре: _____

Допущен к защите
Руководитель:
Доцент, Калитвин Владимир Анатольевич

подпись

Члены комиссии:

учёная степень, учёное звание, ФИО

подпись

учёная степень, учёное звание, ФИО

подпись

учёная степень, учёное звание, ФИО

подпись

Оценка _____

Дата защиты _____

Москва, 2026

МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА
РОССИЙСКОЙ ФЕДЕРАЦИИ

Российский государственный аграрный университет – МСХА
имени К. А. Тимирязева

Институт экономики и управления
Кафедра статистики и кибернетики

**ЗАДАНИЕ
НА КУРСОВОЙ ПРОЕКТ (КП)**

Обучающийся _____

Тема КП _____

Исходные данные к работе _____

Перечень подлежащих разработке в работе вопросов:

Перечень дополнительного материала _____

Дата выдачи задания «___» _____ 202__ г.

Руководитель (подпись, ФИО) _____

Задание принял к исполнению (подпись обучающегося) _____

«___» _____ 202__ г.

Содержание

Введение	5
Глава 1 Теоретическая часть	7
1.1 Информационные системы учета продуктов питания	7
1.2 Анализ чеков и извлечение товарных позиций	7
1.3 Контроль сроков годности	8
Глава 2 Аналитическая часть	10
2.1 Постановка задачи	10
2.2 Функциональные требования	10
2.3 Нефункциональные требования	11
2.4 Выбор архитектурного подхода	11
2.5 Модель данных	12
Глава 3 Практическая часть	13
3.1 Общая характеристика программной реализации	13
3.2 Модуль учета продуктов	13
3.3 Модуль обработки чеков и писем	16
3.4 Модуль базы знаний	20
3.5 Модуль уведомлений	22
Глава 4 Описание алгоритмов	25
4.1 Алгоритм импорта текста чека	25
4.2 Алгоритм классификации продуктовых позиций	25
4.3 Алгоритм подбора срока хранения	26
4.4 Алгоритм ИМАР-импорта писем	26
4.5 Алгоритм формирования уведомлений	27
Глава 5 Тестирование и результаты	28
5.1 Стратегия тестирования	28
5.2 Проверенные сценарии	28
5.3 Результаты практической проверки	29
5.4 Ограничения реализации	29
Заключение	30

Библиографический список	31
Приложения	32
Приложение А. Фрагмент структуры проекта	32
Приложение Б. Пример тестового чека	32

Введение

Современные домашние хозяйства и малые организации сталкиваются с задачей регулярного контроля запасов продуктов питания. При отсутствии систематизированного учета пользователь часто не имеет полной информации о составе запасов, сроках годности и необходимости своевременного использования продуктов. Это приводит к финансовым потерям, росту пищевых отходов и снижению качества планирования покупок. Дополнительную сложность создает то, что сведения о покупке обычно содержатся в кассовых чеках, электронных письмах и сообщениях от магазинов, то есть в слабоструктурированных источниках.

Актуальность темы обусловлена необходимостью разработки прикладной информационной системы, которая объединяет учет продуктов, импорт данных из чеков, определение сроков хранения и уведомление пользователя о приближении даты истечения срока годности. В рамках данной работы рассматривается программный проект Smart Fridge, реализованный на языке Python с использованием фреймворка Django и базы данных PostgreSQL.

Цель курсового проекта – разработать и описать информационную систему учета продуктов питания с анализом чеков и контролем сроков годности.

Для достижения поставленной цели необходимо решить следующие задачи:

- проанализировать предметную область учета продуктов питания и контроля сроков годности;
- определить функциональные и нефункциональные требования к информационной системе;
- описать архитектуру Django-приложения Smart Fridge и структуру его модулей;
- рассмотреть алгоритмы парсинга чеков, классификации продуктовых позиций, подбора срока хранения и формирования уведомлений;
- описать результаты тестирования ключевых сценариев работы системы.

Объектом исследования является процесс учета продуктов питания и контроля сроков годности. Предметом исследования являются методы проектиро-

вания и программной реализации веб-ориентированной информационной системы для автоматизации указанного процесса.

Практическая значимость работы состоит в том, что разработанное приложение может использоваться как прототип персональной системы управления продуктовым запасом, а также как учебный пример построения монолитного веб-приложения на Django с обработкой данных из внешних источников.

Глава 1 Теоретическая часть

1.1 Информационные системы учета продуктов питания

Информационная система учета продуктов питания представляет собой совокупность программных средств, данных и процедур, обеспечивающих регистрацию продуктовых запасов, хранение сведений о количестве, категориях, датах покупки и сроках годности. В отличие от простого списка покупок такая система должна поддерживать жизненный цикл продукта: появление в запасах, уточнение параметров хранения, контроль состояния и удаление после использования.

В предметной области домашнего учета продуктов можно выделить следующие сущности: пользователь, продукт, категория, источник поступления данных, правило срока хранения и уведомление. Пользователь создает или импортирует записи о продуктах. Категория позволяет группировать товары по смысловым признакам, например молочные продукты, хлеб, крупы или напитки. Источник данных показывает, каким способом продукт был добавлен: вручную, из текстового чека, из файла или из письма. Правило срока хранения задает ожидаемую длительность безопасного хранения продукта. Уведомление информирует пользователя о необходимости обратить внимание на продукт.

Для таких систем важны не только операции CRUD, но и качество входных данных. Реальные чеки и письма часто содержат служебные строки, HTML-разметку, сведения о кассе, оплате, рекламе и доставке. Поэтому информационная система должна отделять продуктовые позиции от шумовых данных, нормализовать единицы измерения и предотвращать повторный импорт уже обработанных источников.

1.2 Анализ чеков и извлечение товарных позиций

Кассовый чек является полуструктурированным документом. Он содержит строки товаров, количество, цену, налоговые реквизиты, итоговые суммы и служебную информацию. При обработке бумажных чеков часто используется OCR – оптическое распознавание символов. В результате OCR изображение преобразуется в текст, однако полученный текст может содержать ошибки распозна-

вания, лишние переносы строк и неточные пробелы [9].

В электронных чеках и письмах часть проблем OCR отсутствует, но возникает другая задача: извлечение текста из HTML-писем, фильтрация служебной разметки, определение релевантных писем и защита от повторной обработки. Поэтому в учебной реализации целесообразно использовать комбинированный подход: ручная вставка текста, загрузка файла с последующей возможностью ручной корректировки, а также IMAP-импорт писем.

Для выделения товарных позиций применяются регулярные выражения, словари признаков и простые правила классификации. Регулярное выражение позволяет найти строки вида “название – количество – единица измерения – цена”. Словарь продуктовых слов помогает отличать пищевые товары от непищевых, например исключать пакеты, салфетки, бытовую химию или косметику.

1.3 Контроль сроков годности

Срок годности продукта определяется совокупностью факторов: типом продукта, условиями хранения, датой покупки и индивидуальными свойствами товара. В информационной системе срок хранения может задаваться пользователем вручную либо подбираться автоматически на основе базы знаний. База знаний представляет собой набор правил, где каждому названию или категории продукта соответствует количество дней хранения и условия хранения.

Автоматический подбор срока хранения может выполняться по точному совпадению названия или по оценке схожести. Если точного совпадения нет, система сравнивает название продукта с описаниями и тегами правил. Такая процедура не гарантирует абсолютной точности, поэтому продукты с неопределенным сроком должны получать статус, требующий подтверждения пользователем.

Контроль срока годности реализуется через вычисление даты окончания хранения:

$$D_{exp} = D_{purchase} + L,$$

где D_{exp} – дата истечения срока годности, $D_{purchase}$ – дата покупки, L – срок хранения в днях. На основе этой даты система определяет состояние продукта: свежий, скоро истекает, просрочен или требует подтверждения.

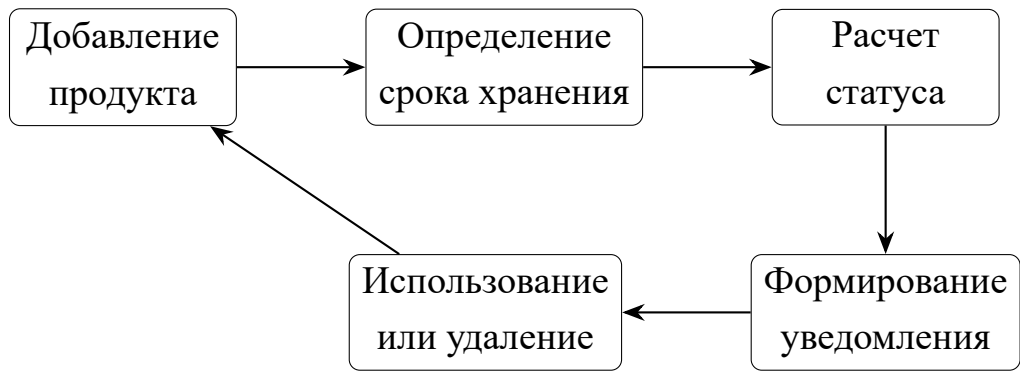


Рис. 1.1 Жизненный цикл записи о продукте в системе

Глава 2 Аналитическая часть

2.1 Постановка задачи

Разрабатываемая система предназначена для автоматизации учета продуктов питания. Пользователь должен иметь возможность регистрироваться, добавлять продукты вручную, импортировать продукты из текста чека или письма, просматривать текущие запасы и получать уведомления о приближении срока годности.

Основная проблема предметной области состоит в том, что данные о покупке продуктов поступают из неоднородных источников. Пользователь может иметь бумажный чек, электронный чек, письмо от магазина или собственный текстовый список. При этом система должна работать без обязательного подключения внешних коммерческих API, чтобы оставаться доступной и воспроизводимой в учебной среде.

2.2 Функциональные требования

Функциональные требования к системе представлены ниже.

- F1. Пользователь должен иметь возможность зарегистрироваться, войти в систему и редактировать профиль.
- F2. Пользователь должен иметь возможность добавлять, редактировать и удалять продукты питания.
- F3. Система должна поддерживать импорт продуктов из текстового представления чека.
- F4. Система должна поддерживать загрузку файла чека и ручное уточнение распознанного текста.
- F5. Система должна поддерживать настройку IMAP-источника электронной почты и импорт писем с чеками.
- F6. Система должна выделять из текста только пищевые товары и игнорировать очевидные непищевые позиции.

- F7. Система должна подбирать срок хранения продукта на основе базы знаний.
- F8. Система должна формировать уведомления за 3 дня, за 1 день и после истечения срока годности.
- F9. Пользователь должен иметь возможность управлять пользовательскими правилами сроков хранения.

2.3 Нефункциональные требования

К нефункциональным требованиям относятся:

- безопасность пользовательских страниц за счет обязательной аутентификации;
- защита HTML-форм от CSRF-атак стандартными средствами Django;
- хранение секретных параметров подключения во внешнем файле окружения;
- устойчивость импорта к ошибкам OCR, отсутствию внешних API и недоступности почтового сервера;
- возможность запуска в локальной среде без Celery, Redis и дополнительных фоновых сервисов;
- совместимость с PostgreSQL как основной реляционной СУБД.

2.4 Выбор архитектурного подхода

Для реализации выбран монолитный подход на Django. Такой подход соответствует масштабу учебного проекта: все основные функции находятся в одном приложении, но разделены по Django-приложениям. Монолит упрощает развертывание, миграции базы данных, тестирование и навигацию по коду. При этом логическое разделение на модули сохраняет возможность последующего расширения.

Основные приложения проекта:

- `accounts` – регистрация, профиль пользователя и настройки уведомлений;
- `pantry` – учет продуктов и категорий;
- `receipts` – обработка чеков, файлов и почтового импорта;
- `knowledge` – база знаний сроков хранения;
- `notifications` – создание и отображение уведомлений.

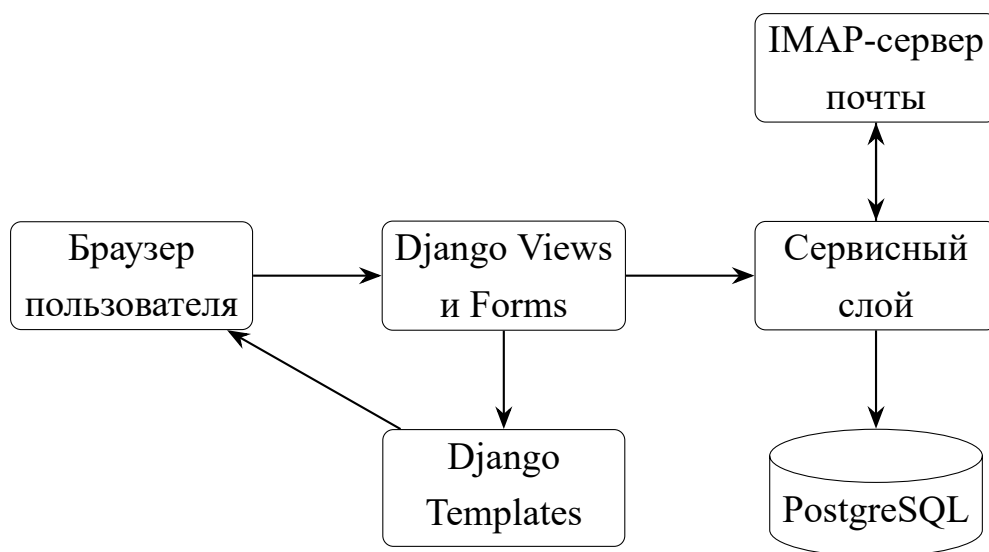


Рис. 2.1 Компонентная архитектура приложения Smart Fridge

2.5 Модель данных

В базе данных используются связанные сущности. Центральной сущностью является `Product`, содержащая название продукта, количество, единицу измерения, категорию, дату покупки, срок хранения, дату истечения и статус. Сущность `Receipt` хранит исходный текст чека или письма, а `ReceiptItem` фиксирует распознанные позиции. Модель `EmailImportSource` задает параметры подключения к ИМАР-серверу, а `ProcessingLog` предотвращает повторный импорт одного и того же письма.

База знаний представлена моделью `ShelfLifeRule`. Правила могут быть общими или принадлежать конкретному пользователю. Уведомления представлены моделью `Notification`; для предотвращения дублей используется уникальность пары “продукт – тип уведомления”.

Глава 3 Практическая часть

3.1 Общая характеристика программной реализации

Проект Smart Fridge реализован на Python 3.11+ с использованием Django 5.x. В качестве СУБД используется PostgreSQL. Пользовательский интерфейс построен на Django Templates, HTML-формах и базовых CSS-стилях. Для работы с электронной почтой используется стандартная библиотека Python `imaplib`; для HTTP-интеграций OCR и LLM предусмотрен модуль `urllib.request`. В проекте сознательно не используются Flask, FastAPI, Django REST Framework, Node.js, React, Vue, Celery, Redis и RabbitMQ.

Структура проекта соответствует типовой организации Django-монолита:

- каталог `smart_fridge` содержит настройки и корневые маршруты;
- каталог `templates` содержит HTML-шаблоны страниц;
- каталог `static/css` содержит базовые стили интерфейса;
- каталог `tests` содержит тесты ключевых пользовательских сценариев.

Практическая проверка выполнялась на запущенном локальном экземпляре приложения. Для демонстрации были подготовлены тестовый пользователь, набор продуктов с различными сроками годности, справочные правила хранения, почтовый источник для импорта чеков и системное уведомление. Такой набор данных позволяет проследить полный пользовательский маршрут: от просмотра содержимого холодильника до анализа чека и контроля приближающихся сроков годности.

3.2 Модуль учета продуктов

Модуль `pantry` реализует основную предметную сущность – продукт. Для продукта задаются название, количество, единица измерения, категория, дата покупки, срок хранения, дата истечения срока годности, статус и источник добавления. Статус продукта вычисляется автоматически при сохранении модели: если дата истечения неизвестна, продукт требует подтверждения; если дата

уже прошла, продукт считается просроченным; если срок истекает в ближайшие три дня, продукт получает статус “скоро истекает”.

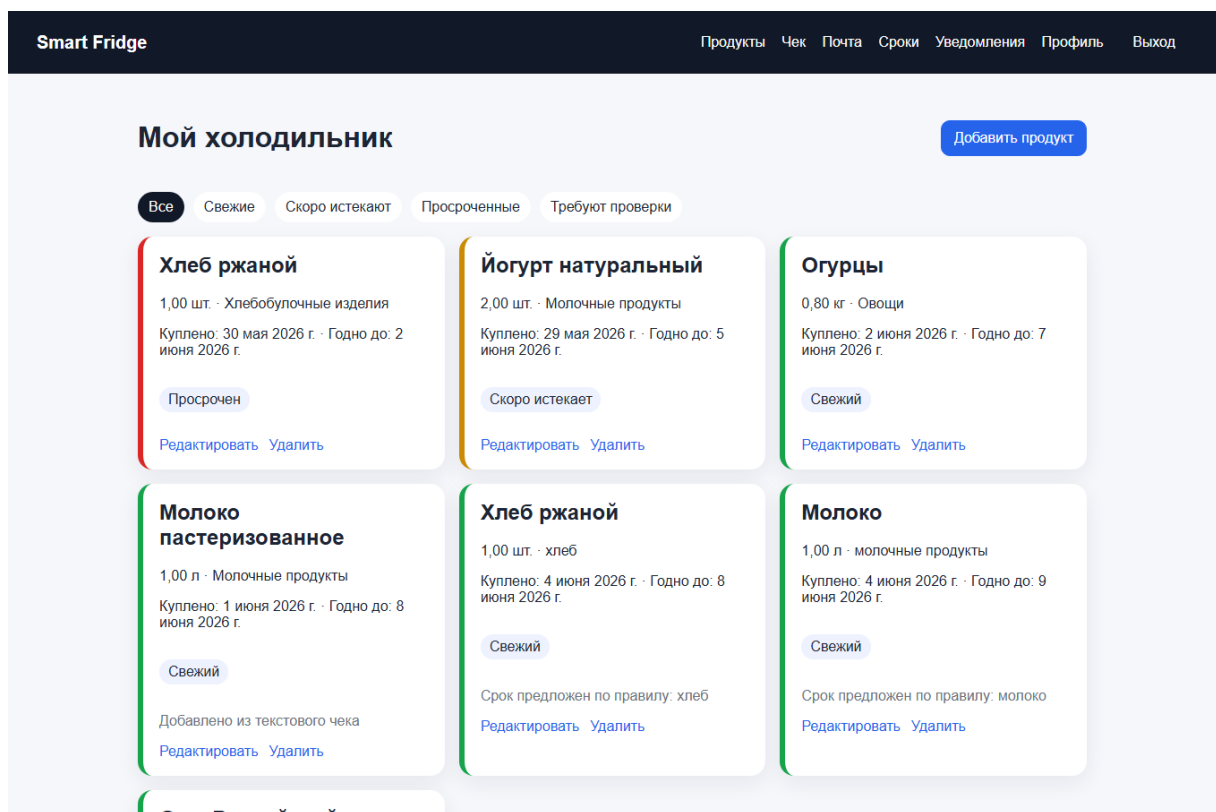


Рис. 3.1 Главный экран учета продуктов с фильтрацией по состоянию сроков годности

Главный экран, представленный на рис. 3.1, является центральной рабочей областью системы. На нем пользователь видит перечень продуктов, сгруппированный визуально в виде карточек. Каждая карточка содержит название, количество, категорию, дату покупки, дату истечения срока годности и вычисленный статус. Цветовая полоса слева от карточки служит дополнительным индикатором состояния: просроченные позиции выделяются красным, продукты с приближающимся сроком – предупреждающим цветом, а актуальные продукты – зеленым.

Над списком расположены фильтры “Все”, “Свежие”, “Скоро истекают”, “Просроченные” и “Требуют проверки”. Эти элементы позволяют быстро перейти от общего перечня к проблемным позициям, требующим внимания пользователя. Кнопка “Добавить продукт” переводит к форме ручного ввода, а ссылки “Редактировать” и “Удалить” на карточках обеспечивают управление отдельными записями без перехода к административной панели Django.

Smart Fridge

Продукты Чек Почта Сроки Уведомления Профиль Выход

Добавить продукт

Название

Количество

Единица

Категория

Дата покупки

Срок годности, дней

Дата истечения

Комментарий

Источник

Рис. 3.2 Форма ручного добавления продукта с параметрами количества, категории и срока хранения

Форма добавления продукта на рис. 3.2 используется в сценариях, когда продукт требуется внести без чека или после ручной проверки распознанных данных. Пользователь указывает наименование, количество, единицу измерения, категорию, дату покупки, срок хранения в днях и при необходимости комментарий. Наличие поля источника позволяет системе различать продукты, добавленные вручную, импортированные из чека или полученные из электронной почты.

Если срок хранения не задан пользователем явно, приложение обращается к базе знаний и пытается подобрать подходящее правило по названию продукта. В результате ручной ввод остается достаточно гибким: пользователь может как полностью контролировать параметры записи, так и использовать автоматическую подсказку срока годности на основе уже накопленных правил.

Листинг 3.1: Фрагмент логики пересчета срока годности

```
def recalculate_expiration(self):
    if self.shelf_life_days is not None:
        self.expiration_date = self.purchase_date + timedelta(days=
            self.shelf_life_days)
```

```
def refresh_status(self):
    if not self.expiration_date or self.shelf_life_days is None:
        self.status = self.STATUS_NEEDS_REVIEW
        return
    today = timezone.localdate()
    if self.expiration_date < today:
        self.status = self.STATUS_EXPIRED
    elif self.expiration_date <= today + timedelta(days=3):
        self.status = self.STATUS_EXPIRING_SOON
    else:
        self.status = self.STATUS_ACTIVE
```

3.3 Модуль обработки чеков и писем

Модуль `receipts` отвечает за получение исходного текста, разбор товарных строк и создание продуктов. В системе предусмотрены три источника: ручной текст, файл чека и электронное письмо. Если внешний OCR-сервис не настроен, файл сохраняется, а пользователь может вручную вставить распознанный текст. Такой fallback повышает надежность учебного проекта.

The screenshot shows a web application interface for 'Smart Fridge'. At the top, there is a dark navigation bar with the application name 'Smart Fridge' on the left and a series of links: 'Продукты', 'Чек', 'Почта', 'Сроки', 'Уведомления', 'Профиль', and 'Выход'. The main content area has a light blue background. In the center, there is a white card titled 'Импорт текста чека'. Inside this card, there is a label 'Текст чека' above a large, empty text input field. Below the input field, there is a blue button with the text 'Разобрать чек'. At the bottom of the card, there is a blue link that says 'Загрузить файл чека'.

Рис. 3.3 Экран ручного импорта текстового представления кассового чека

Экран ручного импорта, показанный на рис. 3.3, предназначен для вставки текста чека, полученного из электронного документа, OCR-сервиса или введенного пользователем вручную. Основным элементом страницы является многострочное поле, в котором каждая строка может соответствовать отдельной товарной позиции. После нажатия кнопки “Импортировать” данные передаются в сервисный слой, где выполняются очистка текста, извлечение количества и единицы измерения, а также фильтрация непищевых позиций.

Данный сценарий важен для практической эксплуатации, поскольку не зависит от внешних API. Даже при отсутствии подключенного OCR пользователь может внести содержимое чека и получить автоматическое создание продуктовых записей. Это снижает количество ручных действий по сравнению с добавлением каждого продукта через отдельную форму.

Smart Fridge

Продукты Чек Почта Сроки Уведомления Профиль Выход

Чек #2 (Текст)

Статус: processed · создан: 4 июня 2026 г. 0:00

Позиции

- Молоко · 1,00 l · молочные продукты → продукт создан
- Хлеб ржаной · 1,00 pcs · хлеб → продукт создан
- Сыр Российский · 300,00 g · молочные продукты → продукт создан

Вставить распознанный текст

Распознанный текст

Молоко 1 л 89.90
Хлеб ржаной 1 шт 54.00
Сыр Российский 300 г 249.00
Пакет 1 шт 7.00

Обработать текст

Рис. 3.4 Результат обработки чека с созданными продуктовыми позициями

После обработки чека приложение открывает страницу результата, представленную на рис. 3.4. На ней фиксируется номер чека, источник данных, статус обработки и дата создания. Ниже отображается перечень распознанных позиций: для каждой строки показаны исходное название, количество, категория и результат создания связанного продукта.

В демонстрационном сценарии система распознала молоко, хлеб и сыр как пищевые товары и создала для них записи в холодильнике. Строка с пакетом используется как пример непищевой позиции, которая не должна превращаться в продуктовый запас. Таким образом, экран результата выполняет контрольную функцию: пользователь может проверить, какие товары были импортированы, и при необходимости повторно обработать распознанный текст.

Почтовый импорт реализован через IMAP. Пользователь задает имя источника, хост IMAP-сервера, порт, логин, пароль приложения, режим SSL и фильтр отправителя или текста. Система проходит по доступным папкам почтового ящика, извлекает текстовые части писем, применяет фильтр и проверяет журнал обработки, чтобы одно письмо не импортировалось повторно.

При обработке письма выполняется очистка текста от недопустимых для PostgreSQL символов, например NUL-байтов. Также система не пытается интерпретировать произвольные бинарные вложения как текст.

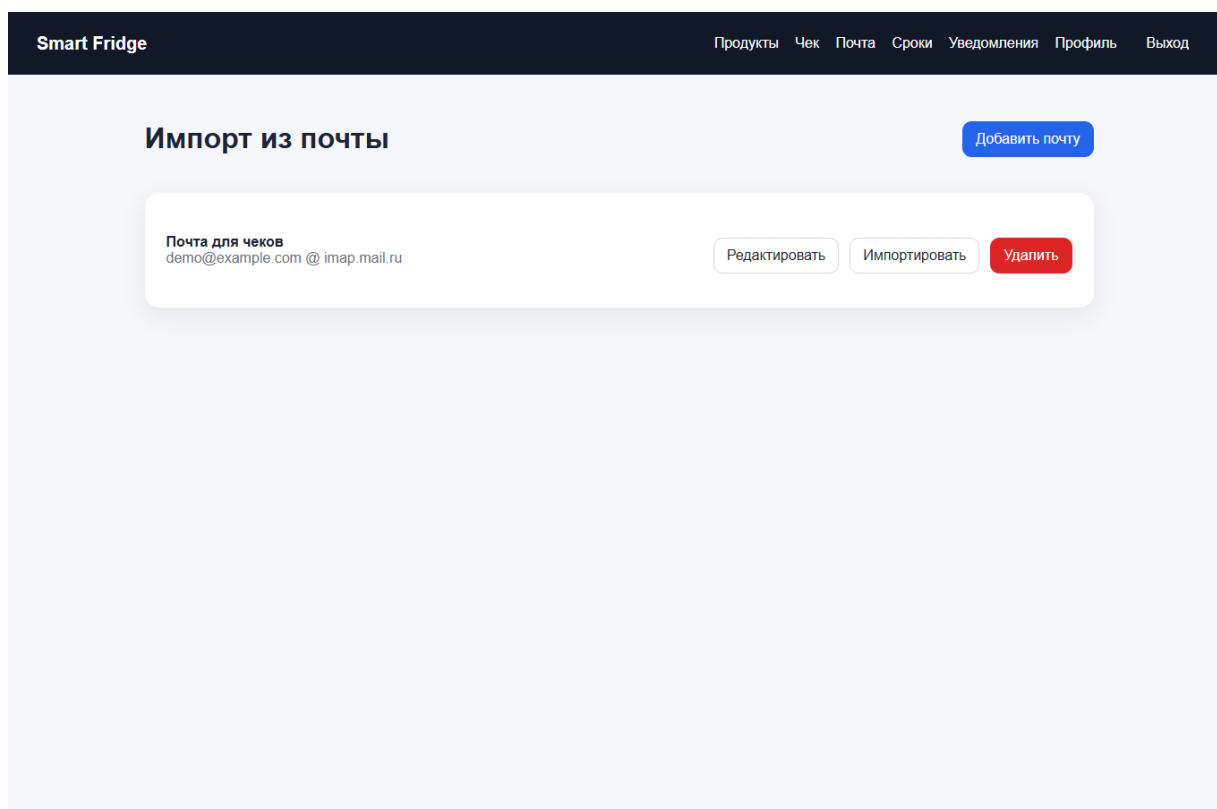


Рис. 3.5 Список IMAP-источников для автоматического импорта чеков из электронной почты

Раздел почтовых источников на рис. 3.5 показывает настроенные подключения к почтовым ящикам. Для каждого источника отображается пользовательское имя, адрес сервера, логин и фильтр, по которому приложение отбирает

письма с чеками. Наличие кнопки запуска импорта рядом с источником делает сценарий управляемым: пользователь может инициировать обработку в нужный момент, не обращаясь к командной строке.

На этом же экране доступны операции редактирования и удаления источника. Такая организация интерфейса важна для безопасности и сопровождаемости: пароль приложения и параметры IMAP можно изменить при смене почтового сервиса, а неактуальные источники удалить, чтобы они не участвовали в последующих попытках импорта.

The screenshot shows a web interface for 'Smart Fridge'. At the top, there is a dark navigation bar with the logo 'Smart Fridge' on the left and a list of menu items: 'Продукты', 'Чек', 'Почта', 'Сроки', 'Уведомления', 'Профиль', and 'Выход'. The main content area is a light blue box titled 'Добавить почту' (Add Mail). Inside this box, there are several form fields: 'Name' with a placeholder 'Основная почта'; 'Host (IMAP-сервер, например: imap.gmail.com, imap.mail.ru, imap.yandex.ru)'; 'Port' with the value '993'; 'Username'; 'Password'; a 'Показать пароль' (Show password) button; 'Use ssl' with a checked checkbox; 'Sender filter (слова или часть адреса отправителя, например: чек, receipt, магазин, @ofd)' with a text input field; a hint 'Например: receipt, чек, магазин'; 'Is active' with a checked checkbox; and two buttons at the bottom: 'Сохранить' (Save) and 'Назад' (Back).

Рис. 3.6 Форма настройки IMAP-подключения и фильтрации писем с кассовыми чеками

Форма добавления почтового источника, показанная на рис. 3.6, содержит параметры, необходимые для подключения к IMAP-серверу: название источника, хост, порт, учетную запись, пароль приложения, признак использования SSL, фильтр отправителя и признак активности. Поле фильтра позволяет ограничить обработку письмами от магазинов, операторов фискальных данных или сервисов электронных чеков.

Практическая ценность этой формы заключается в том, что пользователь может настроить автоматизированный канал пополнения холодильника без изменения программного кода. В учебном проекте импорт запускается синхрон-

но, однако выбранная структура данных допускает дальнейшее развитие сценария, например перенос запуска в планировщик задач или фоновую очередь.

3.4 Модуль базы знаний

Модуль `knowledge` хранит правила сроков хранения. Каждое правило содержит название продукта, описание, срок хранения в днях, условия хранения, категорию, теги и признак активности. Правило может быть общим или пользовательским. Пользовательское правило имеет приоритет над общим, что позволяет адаптировать систему под индивидуальные привычки хранения.

Сервис `ShelfLifeService` сначала ищет точное совпадение названия продукта, затем применяет оценку схожести на основе `SequenceMatcher` и пересечения токенов. При наличии настроенного внешнего LLM-сервиса возможно уточнение срока хранения на основе найденных кандидатов, однако система остается работоспособной без внешнего API.

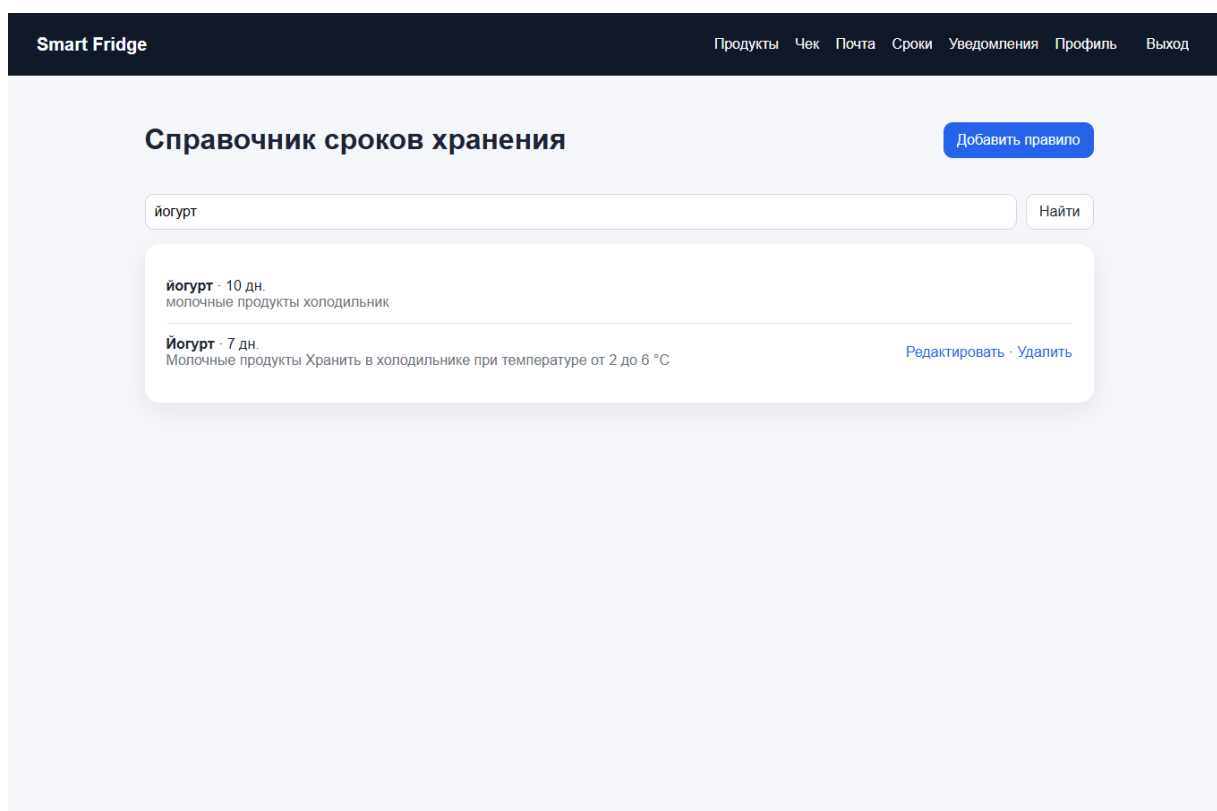


Рис. 3.7 Справочник сроков хранения с поиском правил по названию продукта

Справочник сроков хранения на рис. 3.7 используется для просмотра и поиска правил, по которым приложение предлагает срок годности. В верхней части

находится строка поиска по названию продукта или тегам. В демонстрационном примере выполнена фильтрация по слову “йогурт”, поэтому пользователь видит как общее правило, так и пользовательское правило с индивидуальными условиями хранения.

Наличие нескольких правил по близкому названию показывает механизм приоритета и уточнения. Пользовательская запись может переопределять общий справочник, если в конкретной семье или организации продукт хранится иначе. Это делает систему не только статическим каталогом, но и адаптируемым инструментом поддержки решений.

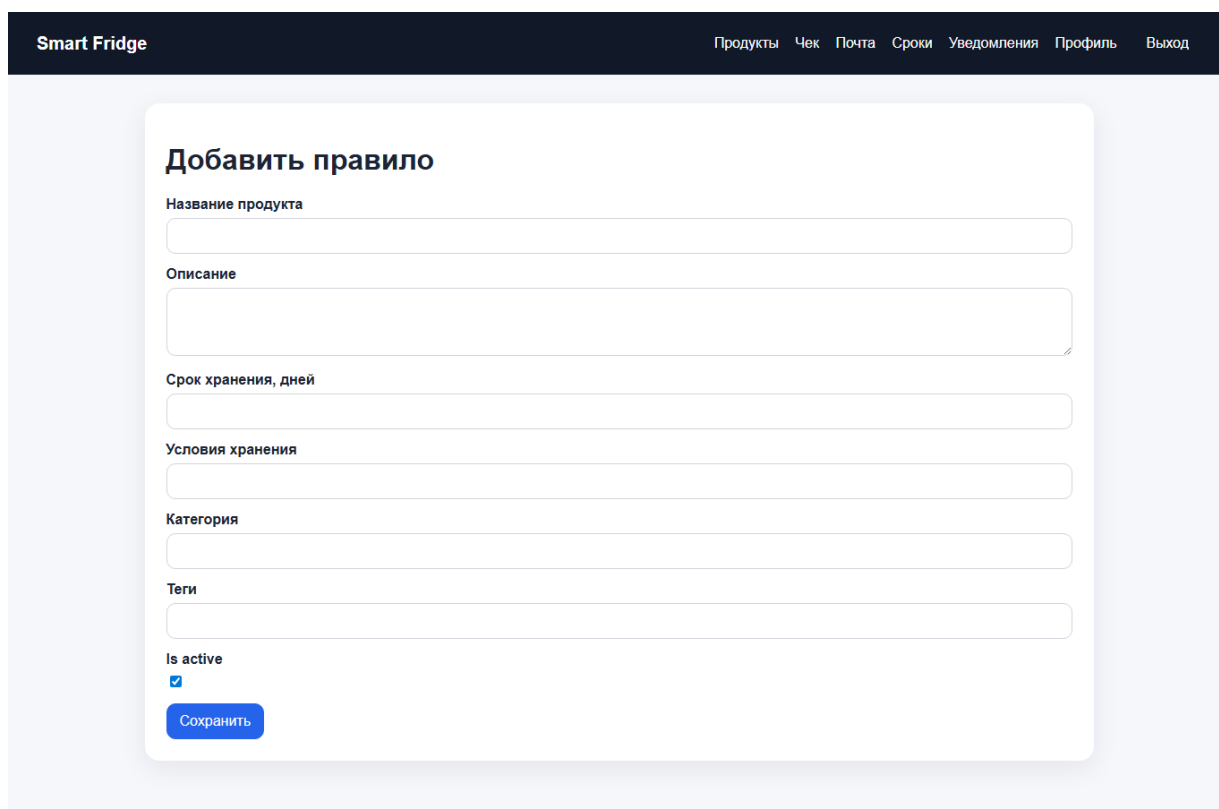
The image shows a web application interface for 'Smart Fridge'. At the top, there is a dark blue header with the text 'Smart Fridge' on the left and a navigation menu on the right containing links: 'Продукты', 'Чек', 'Почта', 'Сроки', 'Уведомления', 'Профиль', and 'Выход'. The main content area is light blue and contains a white rounded rectangle titled 'Добавить правило'. Inside this rectangle, there are several input fields: 'Название продукта', 'Описание', 'Срок хранения, дней', 'Условия хранения', 'Категория', and 'Теги'. Below these fields is a checkbox labeled 'Is active' which is currently checked. At the bottom of the form is a blue button labeled 'Сохранить'.

Рис. 3.8 Форма создания пользовательского правила срока хранения продукта

Форма создания правила, представленная на рис. 3.8, позволяет расширять базу знаний без вмешательства разработчика. Пользователь задает название продукта, описание, срок хранения в днях, условия хранения, категорию, теги и активность правила. Эти поля соответствуют данным, которые затем используются сервисом подбора срока годности при добавлении продуктов.

Особое значение имеют теги и условия хранения. Теги повышают качество поиска и сопоставления похожих названий, а условия хранения объясняют, почему выбран именно такой срок. Благодаря этому приложение не просто автоматически подставляет число дней, а сохраняет контекст принятого решения.

3.5 Модуль уведомлений

Модуль `notifications` формирует уведомления о сроках годности. Сервис проверяет продукты с известной датой истечения и создает уведомления за 3 дня, за 1 день и после наступления просрочки. Если у пользователя включены email-уведомления и указан адрес, сообщение отправляется через стандартный Django email backend.

Формирование уведомлений вынесено в `management command generate_notifications`. Команду можно запускать вручную или по расписанию средствами операционной системы, например через Планировщик задач Windows или `cron`.

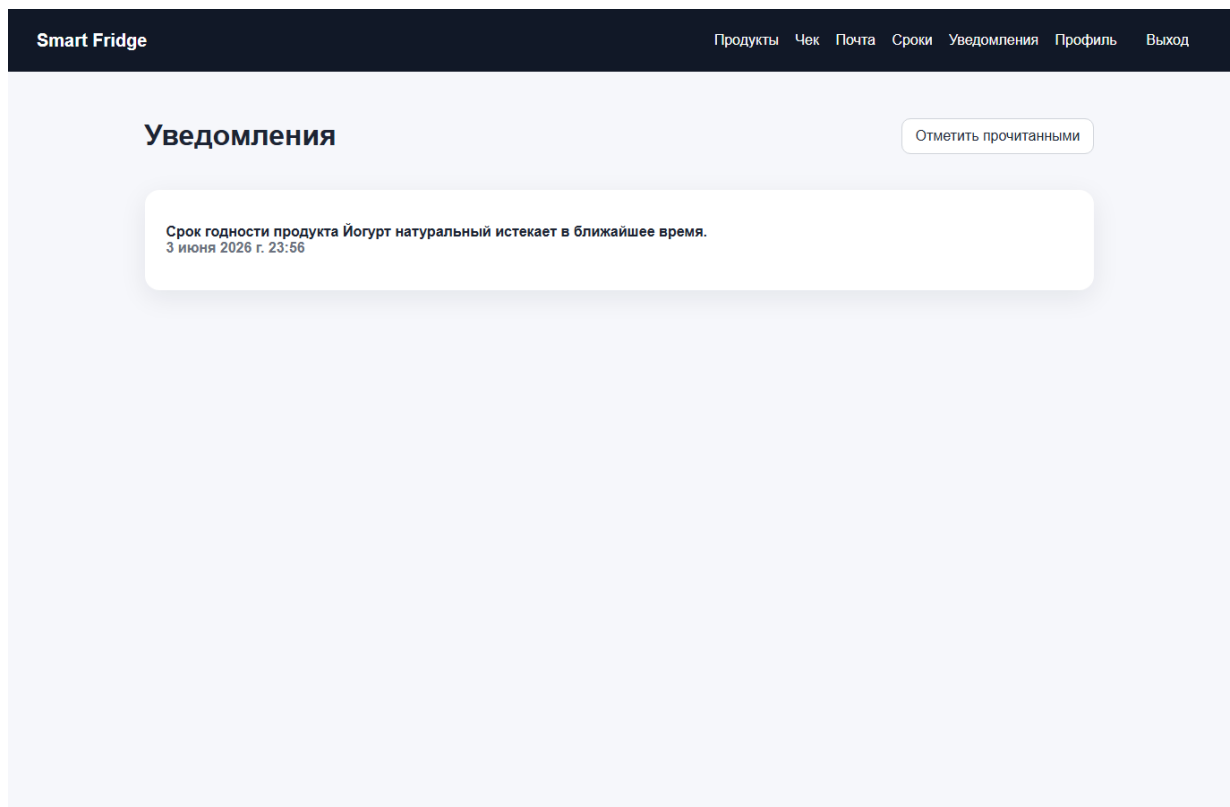


Рис. 3.9 Раздел уведомлений о приближении или наступлении срока годности продуктов

Раздел уведомлений на рис. 3.9 предназначен для концентрации событий, требующих реакции пользователя. В нем отображается текст сообщения, связанный продукт, тип уведомления и признак прочтения. Такой экран дополняет цветовую индикацию на главной странице: даже если пользователь не просматривает весь список продуктов, он может увидеть наиболее важные предупреждения в отдельном разделе.

Кнопка “Отметить все прочитанными” переводит уведомления в обработанное состояние. Это удобно при регулярной работе с системой, поскольку пользователь отделяет новые события от уже просмотренных и тем самым снижает риск пропустить продукт с истекающим сроком годности.

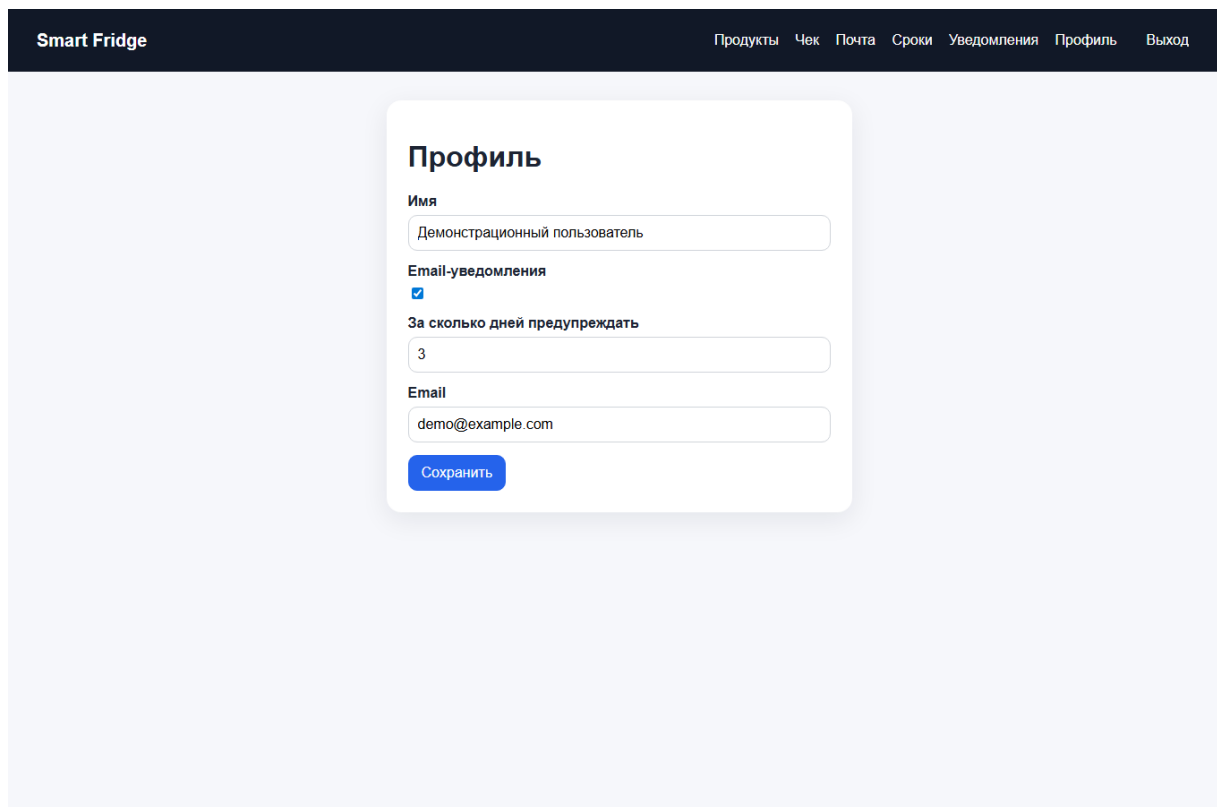


Рис. 3.10 Профиль пользователя с настройками email-уведомлений и горизонта предупреждения

Профиль пользователя, показанный на рис. 3.10, связывает учетную запись с персональными настройками уведомлений. В форме указываются имя, адрес электронной почты, признак включения email-уведомлений и количество дней до истечения срока, за которое пользователь хочет получать предупреждение. Эти параметры позволяют адаптировать поведение системы под разные бытовые сценарии.

Если email-уведомления отключены, приложение сохраняет предупреждения только во внутреннем разделе. Если они включены и почтовый backend настроен, система может отправлять сообщения на указанный адрес. Таким образом, профиль выступает связующим элементом между учетной записью, системой уведомлений и выбранным пользователем способом получения информации.

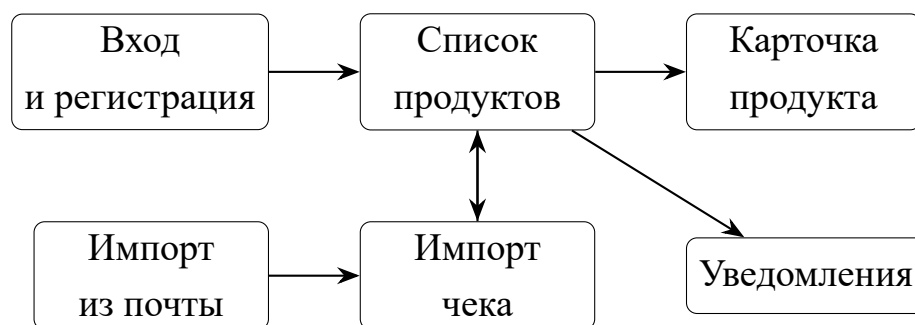


Рис. 3.11 Основные пользовательские экраны системы

Схема на рис. 3.11 обобщает рассмотренные пользовательские маршруты. После входа пользователь попадает к списку продуктов, откуда может перейти к карточкам, импорту чеков, настройке почтового источника, базе знаний и уведомлениям. Такая структура соответствует модульной организации Django-приложения и одновременно сохраняет для пользователя единый навигационный контур.

Глава 4 Описание алгоритмов

4.1 Алгоритм импорта текста чека

Алгоритм импорта текста чека состоит из следующих этапов:

- 1) получение исходного текста из формы, файла или письма;
- 2) удаление недопустимых символов и нормализация строк;
- 3) разбиение текста на отдельные строки;
- 4) исключение пустых строк и служебных строк, содержащих слова “итого”, “кассир”, “ИНН”, “оплата”, “карта”;
- 5) применение регулярного выражения для выделения названия, количества и единицы измерения;
- 6) классификация позиции как пищевой или непищевой;
- 7) создание записи `ReceiptItem`;
- 8) создание записи `Product` для пищевых позиций.

Регулярное выражение допускает русские и латинские символы, числа, проценты, пробелы и типовые единицы измерения. Это позволяет обрабатывать простые строки чеков вида:

Молоко 1 л 99.00

4.2 Алгоритм классификации продуктовых позиций

Классификация основана на словарях ключевых слов. Сначала проверяются непищевые признаки: пакет, салфетки, шампунь, мыло, гель, зубная паста, порошок, батарейка, лампа. Если такие слова найдены, позиция исключается. Далее проверяются пищевые признаки: молоко, кефир, сыр, хлеб, курица, рыба, яйцо, яблоко, картофель, рис, макароны, масло, сок и другие.

Такой алгоритм является простым и интерпретируемым. Его преимущество состоит в прозрачности: пользователь и разработчик могут понять, почему

строка была добавлена или отклонена. Ограничение подхода состоит в зависимости от полноты словаря, поэтому система может быть расширена за счет пользовательских правил или более сложной модели классификации.

4.3 Алгоритм подбора срока хранения

Подбор срока хранения выполняется в несколько этапов. Сначала сервис ищет активное пользовательское правило с точным совпадением названия продукта. Если пользовательского правила нет, выполняется поиск общего правила. Если точного совпадения нет, из базы выбираются кандидаты и вычисляется оценка похожести:

$$S = \max(S_{seq}, S_{tokens}),$$

где S_{seq} – похожесть строк по алгоритму SequenceMatcher, а S_{tokens} – доля пересекающихся токенов между названием продукта и текстовыми полями правила.

Кандидаты с оценкой ниже порога отбрасываются. Если внешний LLM-сервис не настроен, выбирается кандидат с наибольшей оценкой. Если подходящего кандидата нет, продукт получает статус, требующий подтверждения.

4.4 Алгоритм ИМАР-импорта писем

Алгоритм почтового импорта включает подключение к ИМАР-серверу, просмотр папок, получение последних писем, фильтрацию и обработку текста. Для каждого письма формируется внешний идентификатор, состоящий из имени папки и UID письма. Этот идентификатор записывается в ProcessingLog, что предотвращает повторную обработку одного и того же письма.

Листинг 4.1: Упрощенная схема обработки письма

```
for mailbox in mailboxes:
    connection.select(mailbox, readonly=True)
    ids = connection.search(None, "ALL")
    for uid in ids:
        if already_processed(mailbox, uid):
            continue
        message = fetch_message(uid)
        text = extract_text(message)
        if matches_filter(message, text):
```

```
import_text(text, source="email")  
log_success(mailbox, uid)
```

4.5 Алгоритм формирования уведомлений

Для каждого продукта с известной датой истечения вычисляется разница между датой истечения и текущей датой. Если осталось 3 дня или 1 день, создается соответствующее предупреждение. Если дата истечения уже прошла, создается уведомление о просрочке. Уникальное ограничение на пару “продукт – тип уведомления” исключает многократное создание одинаковых сообщений.

Глава 5 Тестирование и результаты

5.1 Стратегия тестирования

Тестирование проекта выполняется средствами стандартного тестового фреймворка Django. Тесты размещены в каталоге `tests` и охватывают ключевые сценарии: регистрацию и вход пользователя, создание продукта, обработку чеков, импорт писем, работу базы знаний и генерацию уведомлений.

Особое внимание уделяется сервисам, так как именно они содержат основную бизнес-логику. Для IMAP-импорта используется мокирование класса `imaplib.IMAP4_SSL`, что позволяет проверять поведение без реального подключения к почтовому серверу.

5.2 Проверенные сценарии

- 1) Парсинг текста “Молоко”, “Пакет”, “Хлеб”: продукты питания добавляются, непищевая позиция исключается.
- 2) Импорт текста чека вручную: создаются записи продуктов для распознанных пищевых строк.
- 3) Очистка текста от NUL-байтов: текст сохраняется в PostgreSQL без ошибки.
- 4) Повторный импорт письма с тем же UID: письмо пропускается, дубли не создаются.
- 5) Редактирование настроек почты: сохраняются новые `host`, `username` и `sender filter`.
- 6) Кириллический фильтр отправителя: фильтр применяется внутри Python без ASCII-ошибок IMAP.
- 7) Импорт из пользовательской папки почты: письмо находится не только во входящих, но и в других IMAP-папках.

5.3 Результаты практической проверки

Практическая проверка показала, что приложение выполняет основные функции учета продуктов. Ручной импорт текстового чека создает продукты из строк “Молоко”, “Хлеб”, “Сыр” и исключает строку “Пакет”. Почтовый импорт способен подключаться к Mail.ru через IMAP при использовании пароля приложения, обрабатывать письма из папки “Чеки” и применять фильтр по отправителю, теме или тексту письма.

В ходе проверки были выявлены и устранены типовые проблемы интеграции: необходимость использования пароля приложения для Mail.ru, различие между адресом почты и IMAP-хостом, обработка кириллических фильтров, чтение писем из пользовательских папок и очистка текста от NUL-символов перед сохранением в PostgreSQL.

5.4 Ограничения реализации

Текущая реализация является учебным прототипом и имеет ряд ограничений. Импорт почты запускается вручную пользователем через кнопку “Импортировать”. Для автоматического запуска требуется добавить отдельную `management command` и настроить ее выполнение по расписанию. Классификация продуктов основана на словарях, поэтому неизвестные названия могут не распознаваться. OCR и LLM являются необязательными расширениями и используются только при наличии соответствующих переменных окружения.

Несмотря на ограничения, архитектура проекта допускает развитие: расширение словаря продуктов, добавление новых правил сроков хранения, подключение OCR-сервиса, автоматизация импорта и улучшение пользовательского интерфейса.

Заключение

В ходе выполнения курсового проекта была рассмотрена и описана информационная система Smart Fridge, предназначенная для учета продуктов питания, анализа чеков и контроля сроков годности. В теоретической части изучены особенности предметной области, задачи обработки чеков и подходы к автоматическому определению сроков хранения. В аналитической части сформулированы функциональные и нефункциональные требования, обоснован выбор монолитной архитектуры на Django и описана модель данных.

Практическая часть показала, что проект реализует основные функции: регистрацию пользователей, учет продуктов, импорт чеков из текста, файлов и электронной почты, ведение базы знаний сроков хранения и формирование уведомлений. Сервисный слой отделяет бизнес-логику от представлений Django, что повышает сопровождаемость системы.

В работе описаны ключевые алгоритмы: разбор текста чека регулярными выражениями, словарная классификация пищевых товаров, подбор срока хранения по базе знаний, IMAP-импорт писем и генерация уведомлений. Тестирование подтвердило работоспособность основных сценариев и устойчивость к ряду практических ошибок, возникающих при обработке электронной почты.

Таким образом, цель курсового проекта достигнута: разработана и описана информационная система учета продуктов питания с анализом чеков и контролем сроков годности. Полученный прототип может быть использован как основа для дальнейшего развития, включая автоматический импорт писем по расписанию, расширение базы знаний и применение более точных методов распознавания товарных позиций.

Библиографический список

1. Django documentation [Электронный ресурс]. – Режим доступа: <https://docs.djangoproject.com/> – Заглавие с экрана. – (Дата обращения: 29.05.2026).
2. PostgreSQL Documentation [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/> – Заглавие с экрана. – (Дата обращения: 29.05.2026).
3. Python 3 Documentation [Электронный ресурс]. – Режим доступа: <https://docs.python.org/3/> – Заглавие с экрана. – (Дата обращения: 29.05.2026).
4. Crispin, M. Internet Message Access Protocol – Version 4rev1 [Электронный ресурс]. – Режим доступа: <https://www.rfc-editor.org/rfc/rfc3501> – Заглавие с экрана. – (Дата обращения: 29.05.2026).
5. ГОСТ Р 7.0.100–2018. Библиографическая запись. Библиографическое описание. Общие требования и правила составления. – Москва: Стандартинформ, 2018. – 124 с. – Текст: непосредственный.
6. Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл. – 10-е изд. – Москва: Вильямс, 2016. – 928 с. – Текст: непосредственный.
7. Коннолли, Т. Базы данных. Проектирование, реализация и сопровождение. Теория и практика / Т. Коннолли, К. Бегг. – 3-е изд. – Москва: Вильямс, 2003. – 1440 с. – Текст: непосредственный.
8. Майерс, Г. Искусство тестирования программ / Г. Майерс, Т. Баджетт, К. Сандлер. – 3-е изд. – Москва: Диалектика, 2012. – 272 с. – Текст: непосредственный.
9. Гонсалес, Р. Цифровая обработка изображений / Р. Гонсалес, Р. Вудс. – 3-е изд. – Москва: Техносфера, 2012. – 1104 с. – Текст: непосредственный.

Приложения

Приложение А. Фрагмент структуры проекта

smart_fridge/	
accounts/	registration and profile
pantry/	products and categories
receipts/	receipt parsing and email import
knowledge/	shelf-life rules
notifications/	expiration notifications
templates/	Django templates
tests/	key scenario tests

Приложение Б. Пример тестового чека

Молоко 1 л	99.00
Хлеб 1 шт	55.00
Сыр 1 шт	180.00
Пакет 1 шт	7.00